Geospatial Web Lab

Architecture, setup, and verification | 29 September 2026

Five full-stack learning applications, each with its own Rails backend, React frontend, PostgreSQL/PostGIS database, API, tests, and Docker Compose configuration. The root npm workspace shares map, chart, authentication, and layout components without sharing application data.

This is an **AI-assisted learning and portfolio project using synthetic application data**. Local verification uses real APIs, databases, and workers. Container builds and startup are verified in CI. The standalone browser editions are published at https://efkopru.github.io/geospatial-web-lab/; the full-stack applications are not publicly hosted. Remote CI results are reported separately in GitHub Actions.

Documentation

Document	Purpose
README	Project overview, architecture, setup reference, and verification boundaries
How to use	Complete prerequisites, permissions, five application workflows, tests, and recovery
Project summary	Evidence-based portfolio narrative and copy-ready project description

Matching PDF editions are in `output/pdf/`. Detailed evidence remains in VERIFICATION.md, AUDIT.md, and AUDIT_SECOND_PASS.md. Application-specific READMEs are linked below.

Edit the Markdown sources first, then rebuild all three PDFs with `python scripts/build-docs.py`. The builder requires Python and ReportLab.

Ten local application screenshots are included in `output/screenshots/`, including mobile layouts. Service request details and fleet replay show synthetic workflows. Third-party reference captures and duplicate screenshot ZIPs remain local.

Applications

Application	UI / API ports	Main workflow
01 Service requests	5171 / 3101	Report, assign, resolve; live updates; queued import/export
02 Data quality	5172 / 3102	Upload GeoJSON, validate, review, approve immutable export
03 Fleet monitor	5173 / 3103	Replay ten simulated vehicles, monitor geofence transitions
04 Parcel scenarios	5174 / 3104	Select parcels, calculate assumptions, compare saved scenarios
05 Infrastructure inspections	5175 / 3105	Inspect 3D assets, record observations, generate elevation profiles

Open each frontend at `http://127.0.0.1:PORT`, using its UI port. Development servers bind to loopback.

All application datasets are synthetic. The map basemap is an external Esri service. Cesium's default corridor uses synthetic elevations and requires no access token. Every application persists records in its own database; browser storage is not used as a database substitute.

Architecture and learning coverage

```
React interface + ArcGIS or Cesium
    | HTTP requests and authenticated WebSocket updates
Rails API + ActionCable
    |                          |
PostgreSQL + PostGIS      Redis + Sidekiq
    |                          |
Persistent records      Queued domain jobs
```

Each application is an independent stack. The UI reads authoritative records from Rails. ActionCable notifications prompt refreshes; reconnect handling and polling recover missed updates. When Redis restarts, each API keeps retrying its Redis subscription and resubscribes open streams once Redis is back (`config/initializers/action_cable_redis_listener.rb`). Rails authorizes writes, and Sidekiq processes imports, exports, validation, replay, and calculations. Durable results are independent of notification delivery.

Learning objective	Implemented evidence
Rails, JavaScript, and React	Authenticated forms, JSON APIs, server validation, and account-specific workflows
Interactive UI and visualization	Linked maps/tables, charts, live updates, playback, comparisons, and 3D controls
ArcGIS, CesiumJS, and PostGIS	Spatial validation, distance/area queries, geofences, 2D feature interaction, and 3D assets
Backend and job processing	Spatial indexes, batched spatial queries, bounded results/history, visible job states, bounded automatic retries, manual recovery, deduplication, and concurrency guards
Integration and deployment preparation	Proxies, WebSockets, migrations, health checks, runtime scripts, and container configuration
Automated reliability	Domain, permissions, browser workflows, persistence, and process lifecycle regressions

The local verification record uses Rails 8.1.4, React 19.3, ArcGIS Maps SDK 5.1.26, CesiumJS 1.145, and Sidekiq 7.3.9. Lockfiles specify exact dependency versions. These implementations demonstrate techniques; they do not establish production scale, uptime, or business impact.

Local accounts

After seeding, sign in with `staff@example.test` or `reporter@example.test`, password `Learning123!`. These are learning accounts only. Staff can manage operational workflows; reporters have restricted access enforced by Rails. Each app uses a different session cookie.

Sessions expire after 24 hours and can be revoked on the server. Failed sign-ins are limited to five for one email from one client address and twenty from one address within 15 minutes, and to thirty for one email from all addresses within an hour. A browser that has signed in to an account before is exempt from the hourly limit, so guessing from elsewhere cannot lock the owner out of a browser they already use; an encrypted cookie remembers the five accounts it signed in to most recently, for a year after the last sign-in. Successful sign-ins are not counted. Logout invalidates copied cookies and disconnects that login's live subscriptions. If notification delivery is unavailable, subscriptions recheck authorization every 15 seconds. The session migration invalidates cookies from older application revisions; sign in again after upgrading.

Login and logout changes propagate to other tabs for the same application. Account changes discard the previous account's drafts. Returning to a tab rechecks its session while preserving drafts when the account is unchanged.

Local setup reference

The verified environment uses Node 24 on Windows, with Ruby 3.4.11, PostgreSQL 16/PostGIS 3.4, and Redis 7 in Ubuntu 24.04 WSL. Install prerequisites and backend gems as described in HOW_TO_USE.md before this sequence. Bootstrap configures services and databases; it does not install the development toolchain.

From PowerShell in this folder:

```
npm ci
$project = (Get-Location).Path
$wslProject = (wsl -d Ubuntu -- wslpath -a "$project").Trim()
wsl -d Ubuntu -u root -- bash "$wslProject/scripts/bootstrap-wsl.sh"
# Explicit first-time demo setup; startup never seeds or resets data.
wsl -d Ubuntu -u root -- bash "$wslProject/scripts/seed-demo.sh"
```

Keep two terminals open:

```
# Terminal 1: migrations, APIs, and workers
wsl -d Ubuntu -u root -- bash "$wslProject/scripts/start-backends.sh"
```

```
# Terminal 2: all five React frontends
node scripts/dev-frontends.mjs
```

`npm run dev:1` through `npm run dev:5` start individual frontends. Both combined launchers stop their child processes on Ctrl+C or if one child fails. All local servers bind to loopback. Do not mix combined and individual launchers on the same ports. To reload backend code and workers, stop the existing supervisor, then run the startup command again. To stop Rails and workers from another terminal, run:

```
wsl -d Ubuntu -u root -- bash "$wslProject/scripts/stop-backends.sh"
```

The application processes use the `geolab` database role with password `geolab`. The local database role is not a superuser. Bootstrap creates PostGIS as the PostgreSQL administrator, and migrations retain that extension. PostgreSQL and Redis services remain available until stopped separately.

Set up another Linux/WSL environment

Install Ruby 3.4 or newer (the Gemfiles require it; CI and the supplied container use 3.4, and Ubuntu 24.04 packages only 3.2), Bundler, PostgreSQL with matching PostGIS packages, Redis, Node 24, and native build prerequisites (`build-essential`, `libpq-dev`, `libyaml-dev`). Run `bundle install` in each backend. Run bootstrap as root on Ubuntu, or provision the databases and extensions with your database administrator. Adjust the `/mnt/c/...` path when using a different checkout.

Application database names are `service_requests`, `data_quality_portal`, `fleet_monitor`, `parcel_scenarios`, and `infrastructure_inspections`; test databases add `_test`. Override `DATABASE_URL`, `TEST_DATABASE_URL`, and `REDIS_URL` to use other services.

Container deployment

Each project contains `compose.yaml` and `.env.example`. From the chosen project folder:

```
node ../scripts/configure-env.mjs
# Generates unique session, administrator, and application database passwords.
# Existing secrets are preserved.
docker compose up --build -d
# Optional: creates synthetic data and known learning accounts.
docker compose exec api bundle exec rails db:seed
```

Compose builds the selected frontend from the parent npm workspace. It runs nginx, Rails/Puma, a separate Sidekiq worker, PostGIS, and Redis. Database and Redis volumes persist across container restarts. Services restart after failures. The Rails image runs as a non-root user. PostgreSQL initializes PostGIS as administrator and grants a separate `geolab_app` role ownership of application tables; Rails never receives the administrator password. Migrations finish before API and worker startup and do not create demo accounts. Application ports bind only to localhost by default. nginx sends security headers, including a Content-Security-Policy that allows scripts only from the site and map data only from the ArcGIS hosts (the comment in `scripts/nginx.conf` explains each source). It compresses text assets, caches Vite's content-hashed files for a year, and revalidates `index.html` on every visit. The sign-in form prefills the learning accounts unless `.env` sets `DEMO_ACCOUNTS=false` before the build; use that setting for a stack that is not seeded. Use all five Compose files to run all five independent stacks.

The backend image pins Ruby 3.4 on Debian trixie and explicitly installs the PostgreSQL 17 client to match the database service. CI also installs client 17; PostgreSQL's dump utility cannot read a server with a newer major version than the client.

`scripts/container-smoke.sh <project>` builds one stack with generated secrets and starts it as a separate Compose project (`geolab-smoke-<project>`), so it never touches the containers or volumes of your own stack for that app. It uses the app's usual port, so stop your own stack for that app first; the script stops early if the port is busy. It then checks that migrations finished and the API is healthy. Through nginx, it requests the frontend bundle, `/up` and `/api/session`. It also runs `db:seed` in the API container, waits for a Sidekiq process to register in Redis, and confirms that the API, worker and web containers have not restarted. It removes the stack and its volumes afterwards unless `--keep` is passed. The container workflow runs it for all five projects when backend, frontend or container files change. `scripts/fullstack-benchmark.mjs <project>` measures API latency, concurrent reads and background-job duration against a running stack; the benchmark workflow runs it in CI, and results are in VERIFICATION.md.

For a volume created by an older revision, first generate the new `APP_DATABASE_PASSWORD`, then run `docker compose up -d db` and `docker compose exec db bash /docker-entrypoint-initdb.d/20-geolab.sh` before starting the full stack. This idempotent administrator step grants the new application role ownership of existing application tables while leaving PostGIS extension objects with the administrator. Back up the database first; do not delete a volume to upgrade it.

For an external deployment, provision HTTPS, set `ALLOWED_HOSTS` and `ALLOWED_ORIGINS` to the real host, enable `SECURE_COOKIES`, configure trusted TLS termination and `FORCE_SSL` consistently, replace demo credentials and build the frontend with `DEMO_ACCOUNTS=false`, and keep secrets outside source control. The nginx configuration handles same-origin API requests and WebSocket upgrades. The full-stack applications have no public hosting.

Verification

The application audit dated **2026-09-29** recorded 109 backend tests with 690 assertions, 44 frontend tests across 10 suites, 13 Playwright scenarios against real services, and five successful frontend builds. All five Compose configurations parsed successfully. These are recorded results, not new test runs performed while authoring the documentation. A local run on **2026-10-17**, after the October review fixes, the fixes for issues #16–#22 and the profile-run recovery, recorded 159 backend tests with 1126 assertions, 58 root frontend tests across 13 files, and 71 standalone domain and storage tests plus 10 interface tests; ESLint reported no problems and Brakeman no warnings. See VERIFICATION.md.

```
npm test
npm run lint
node --test tests/runtime-scripts.node.mjs
node --test tests/shared-copies.node.mjs
npm run build
wsl -d Ubuntu -u root -- bash "$wslProject/tests/runtime-shell.test.sh"
wsl -d Ubuntu -u root -- bash "$wslProject/scripts/test-backends.sh"
# With all APIs, workers and frontends running:
npx playwright install chromium
npm run test:e2e
```

The lint workflow also runs Brakeman, which is not in the Gemfiles. To run it locally, install `brakeman` 8.1.0 and run `brakeman --no-pager <project>/backend` for each backend. Every check runs, including the end-of-life Ruby check against the Ruby 3.4 the lockfiles record.

Backend suites exercise PostGIS operations, validation, ownership and staff permissions, state transitions, replay generation guards, job deduplication, immutable dataset versions, stale writes, and calculated outcomes. React tests verify user interactions and the shared request/CSRF contract. Browser tests use real APIs and workers. One serves each production build with the nginx Content-Security-Policy and fails on anything the policy blocks; the Vite development servers send no policy. Read VERIFICATION.md for observed results and remaining environmental limitations.

Tests explicitly migrate their test databases. Rails automatic test schema replacement is disabled because a restricted app role must not drop/recreate the administrator-owned PostGIS extension. `db/structure.sql` captures native spatial/generated columns and database constraints.

Browser tests add synthetic records to development databases. Docker image builds and container runtime could not be verified locally because the engine could not start. The container workflow builds and smoke-tests each stack on GitHub Actions; all five passed on October 1, 2026 (see VERIFICATION.md) and again on `main` at f4eb28b on October 5, 2026. The GitHub workflow repeats validation on pull requests and `main`; its run status is separate from the dated local results above. The lint workflow runs ESLint, Brakeman, and a check that intentionally copied files are still identical on every change. The full-stack applications have no public deployment; only the standalone browser editions are published (see standalone/README.md). Successful builds and Compose parsing do not establish deployment readiness.

Backup and restore

From a project's folder, replace `DATABASE_NAME` with its database name and run:

```
docker compose exec -T db pg_dump -U geolab -d DATABASE_NAME -Fc `
  --exclude-table-data=public.spatial_ref_sys -f /tmp/backup.dump
docker compose cp db:/tmp/backup.dump ./backup.dump
```

Writing the dump inside the container and copying it preserves binary bytes on every PowerShell version. Restore into a separately created database using `pg_restore --no-owner --no-acl`; provision PostGIS with the database administrator first. The dump excludes PostGIS's administrator-owned `spatial_ref_sys` data because the extension provides those reference rows. Do not overwrite a working database as a restore test. A local backup/restore verification script is provided at `scripts/verify-backup.sh`.

Where to learn

- React state and API integration: each `frontend/src/App.jsx`; shared `api` handles cookies, CSRF and errors.
- Spatial backend design: each `backend/db/migrate/002_create_domain.rb`, model/service SQL, and GiST indexes. The fleet replay (`replay_engine.rb`, `telemetry_recorder.rb`) batches its distance and geofence reads. Adding geofences does not add spatial-read queries per frame, but new memberships and transitions require individual writes.

- Real-time delivery: `backend/app/channels`, model/service broadcasts, and shared `useLive`. Reconnect triggers a fresh authoritative fetch; polling provides recovery.
- Background processing: `backend/app/jobs`, Sidekiq workers, visible processing states, retry/idempotence guards. Active Job allows up to three attempts for imports and exports, and retries database-connection failures during validation. Failed imports, validations, and scenarios can be retried in the app; failed exports are regenerated. These jobs disable Sidekiq's additional automatic retries.
- Deployment: project Compose files, backend Dockerfiles, `scripts/Frontend.Dockerfile`, nginx proxy, and CI workflow.
- Reliability: backend domain tests, `tests/*.test.jsx`, and `tests/browser` workflows.

Portfolio boundaries

Use PROJECT_SUMMARY.md with the dated verification record. Identify synthetic data and the revision shown in screenshots or demonstrations. Parcel formulas and height warnings are planning exercises; interpolated corridor profiles are not measured terrain or engineering surveys. Fleet movement is simulated, with no guaranteed wall-clock delivery rate.

WebGL and external ArcGIS basemap/asset access are required for full map rendering. No load-test result, independent security certification, real-world adoption, or business outcome is claimed. Original project material is published with all rights reserved; public source does not grant an additional reuse or redistribution license. Third-party software, basemaps, and assets retain their own licenses and terms; see the preserved notices.